Program Name	**B.C.A**	Semester	**VI**
Course Title	**Fundamentals of Data Science (Theory)**		
Course Code:	**DSE-E2**	No. of Credits	**03**
Contact hours	**42 Hours**	Duration of SEA/Exam	**2 1/2 Hours**
Formative Assessment Marks	**40**	Summative Assessment Marks	**60**

Course Outcomes (COs): After the successful completion of the course, the student will be able to:

CO1	Understand the concepts of data and pre-processing of data.
CO2	Know simple pattern recognition methods
CO3	Understand the basic concepts of Clustering and Classification
CO4	Know the recent trends in Data Science

Contents	**42 Hrs**
Unit I: Data Mining: Introduction, Data Mining Definitions, Knowledge Discovery in Databases (KDD) Vs Data Mining, DBMS Vs Data Mining, DM techniques, Problems,Issues and Challenges in DM, DM applications.	**8**
Data Warehouse: Introduction, Definition, Multidimensional Data Model, Data Cleaning, Data Integration and transformation, Data reduction, Discretization	**8**
Mining Frequent Patterns: Basic Concept – Frequent Item Set Mining Methods - Aprioriand Frequent Pattern Growth (FPGrowth) algorithms -Mining Association Rules	**8**
Classification: Basic Concepts, Issues, Algorithms: Decision Tree Induction. Bayes Classification Methods, Rule-Based Classification, Lazy Learners (or Learning from yourNeighbors), k Nearest Neighbor. Prediction - Accuracy- Precision and Recall	**10**
Clustering: Cluster Analysis, Partitioning Methods, Hierarchical Methods, Density-Based Methods, Grid-Based Methods, Evaluation of Clustering	**8**

Unit 3

Topics:

Mining Frequent Patterns: Basic Concept – Frequent Item Set Mining Methods -Apriori and Frequent Pattern Growth (FPGrowth) algorithms -Mining Association Rules.

Basic Concepts

Item: Refers to an item/product/data value in a dataset. E.g., Mobile, Case, Mouse, Keyboard, etc.

Itemset: Set of items in a single transaction. Eg., X={Mobile, charger, screen guard} Y={Headset, insurance};

Frequent Itemset: Set of itemset occurring repeatedly/frequently in a dataset (i.e. in many transactions).

X={X1,X2,X3, ….., Xk}

k-itemsets

Closed Itemset: An itemset is closed in a data set if there is no superset that has the same support count as the original itemset.

For example, if a dataset contains 100 transactions and the item set {milk, bread} appears in 20 of those transactions, the support count for {milk, bread} is 20. If there is no superset of {milk, bread} that has a support count of 20, then {milk, bread} is a closed frequent itemset.

Closed frequent itemsets are useful for data mining because they can be used to identify patterns in data without losing any information. They can also be used to generate association rules, which are expressions that show how two or more items are related.

Support: It is a measure of frequency of items occurring in a dataset. It is the probability that a transaction contains the item. It is calculated by dividing the number of transactions containing the item(s) by the total number of transactions in the dataset. For example, if an itemset occurs in 5% of the transactions in a dataset, it has a support of 5%. Support is often used as a threshold for identifying frequent item sets in a dataset, which can be used to generate association rules. For example, if we set the support threshold to 5%, then any itemset that occurs in more than 5% of the transactions in the dataset will be considered a frequent itemset.

Support(X) = (Number of transactions containing X) / (Total number of transactions)

where X is the itemset for which you are calculating the support.

Support(X -> Y) = Support_count(X ∪ Y)

Confidence:

Confidence is a measure of the likelihood that an itemset will appear if another itemset appears. It is based on conditional probability. IT is measure For example, suppose we have a dataset of 1000 transactions, and the itemset {milk, bread} appears in 100 of those transactions. The itemset {milk} appears in 200 of those transactions. The confidence of the rule “If a customer buys milk, they will also buy bread” would be calculated as follows:

Confidence("If a customer buys milk, they will also buy bread")

= Number of transactions containing

{milk, bread} / Number of transactions containing {milk}

= 100 / 200

= 50%

Confidence(X => Y) = (Number of transactions containing X and Y) / (Number of transactions containing X)

Confidence(X -> Y) = Support_count(X ∪ Y) / Support_count(X)

Support and confidence are two measures that are used in association rule mining to evaluate the strength of a rule. Both support and confidence are used to identify strong association rules. A rule with high support is more likely to be of interest because it occurs frequently in the dataset. A rule with high confidence is more likely to be valid because it has a high likelihood of being true.

Association Rule:

Association rules are "if-then" statements, that help to show the probability of relationships between data items, within large data sets in various types of databases. Frequent patterns are represented by association rules.

X=> Y

If(antecedent)=> Y(Consequent)

Buys (X,”Laptop”) => Buys (Y,”Wireless Mouse”) [Support=50%, Confidence=70%]

Frequent Pattern (Itemset) Mining:

Frequent pattern mining in data mining is the process of identifying patterns or associations within a dataset that occur frequently. This is typically done by analyzing large datasets to find items or sets of items that appear together frequently.

Importance of Frequent Pattern Mining:

It helps to find association, correlation and interesting relationship among data.

In general, association rule mining can be viewed as a two-step process:

1. Find all frequent itemsets: By definition, each of these itemsets will occur at least as frequently as a predetermined minimum support count, min sup.

2. Generate strong association rules from the frequent itemsets: By definition, these rules must satisfy minimum support and minimum confidence.

Applications of Frequent Pattern Mining:

- **Market basket analysis:** Helps identify items that are commonly purchased
- **Web usage mining:** Helps understand user browsing patterns
- **Bioinformatics:** Helps analyze gene sequences
- **Fraud detection:** Helps identify unusual patterns
- **Healthcare:** Analyzing patient data and identifying common patterns or risk factors.
- **Recommendation systems:** Identify patterns of user interaction and helps with recommendation to the users of an application.
- **Cross-selling and up-selling** : Identifying related products to recommend or suggest to customers.

Frequent Itemset Mining Methods

Methods for mining the simplest form of frequent patterns.

1. Apriori Algorithm
2. Frequent Pattern Growth Mining
3. Vertical Data Format Method

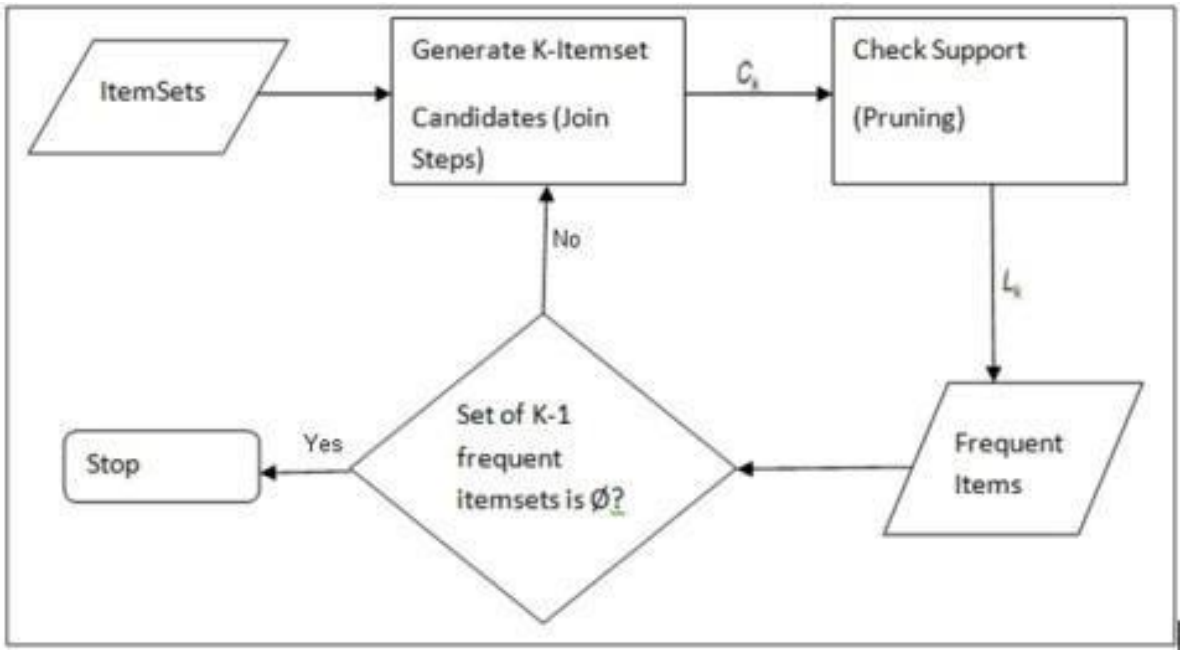
Apriori Algorithm:

Apriori is a important algorithm proposed by R. Agrawal and R. Srikant in 1994. It is uses frequent itemsets to generate association rules. It is based on the concept that a subset of frequent itemset must also be frequent itemset, which is an Apriori property. For example, if the itemset {A, B, C} frequently appears in a dataset, then the subsets {A, B}, {A, C}, {B, C}, {A}, {B}, and {C} must also appear frequently in the dataset. It is an iterative technique which use breadth first search strategy to discover repeating groups/pattern.
It contains two steps:

1. Join Step: Find the itemsets (L_k)
2. Prune Step: Remove the itemsets in which sub items do not satisfy the min support count threshold.

Technique:

1. Set the minimum support threshold - min frequency required for an itemset to be "frequent".
2. Identify frequent individual items - count the occurence of each individual item.
3. Generate candidate itemsets of size 2 - create pairs of frequent items discovered.
4. Prune infrequent itemsets - eliminate itemsets that do no meet the threshold levels.
5. Generate itemsets of larger sizes - combine the frequent itemsets of size 3,4, and so on.
6. Repeat the pruning process - keep eliminating the itemsets that do not meet the threshold levels.
7. Iterate till no more frequent itemsets can be generated.
8. Generate association rules that express the relationship between them - calculate measures to evaluate the strength & significance of these rules.



Algorithm:

variable C_k: candidate itemsets of size k
variable L_k: frequent itemsets of size k

L_1 = {frequent items}
for (k = 1; L_k !=∅; k++) **do begin**
    // JOIN STEP: join L_k with itself to produce C_{k+1}
    // PRUNE STEP: discard (k+1)-itemsets from C_{k+1} that contain non-frequent k-itemsets as subsets
    C_{k+1} = candidates generated from L_k

    for each transaction t in database do
        Increment the count of all candidates in C_{k+1} that are contained in t
    L_{k+1} = candidates in C_{k+1} with min_support
return $\cup_k L_k$

Example:

Consider a dataset of simple business transactions: Min support=50% and Threshold confidence=70%

TID	Items
100	1,3,4
200	2,3,5
300	1,2,3,5
400	2,5

Where TID referes to Transaction ID and 1,2,3.. refers to items/products(for simplicity numbers are considered)

Step 1: finding repeating individual items and counting its occurrences(support) using the formula and consider it as C1 (size 1).

Support(X) = (Number of transactions containing X) / (Total number of transactions)

Item	**Support**
1	2/4=50%
2	¾=75%
3	¾=75%
4	¼=25%
5	¾=75%

Remove the items which has support less than 50%.

Itemset- L1
1
2
3
5

Step 2: Form Itemset of size 2 (pairs) by using L1.

Item	**Support**
1,2	1/4=25%
1,3	2/4=50%
1,5	1/4=25%
2,3	2/4=50%
2,5	3/4=75%
3,5	2/4=50%

Remove the items which has support less than 50%.

Itemset L2

Itemset
1,3
2,3
2,5
3,5

Step 3: Form Itemset of size 3 (triplets) by using L2.

Item	Support
1,2,3	1/4=25%
1,3,5	1/4=25%
1,2,5	1/4=25%
2,3,5	2/4=50%

Remove the items which has support less than 50%.

Note: {1,2} has already been eliminated in step 2 therefore as per Apriori principle no need to consider in this step.

Itemset L3

Itemset
2,3,5

As no more itemset of size 4 can be generated therefore stop the iteration.

Now compute the support and confidence for the generated association rules for itemset {2,3,5}.

Confidence is computed using the formula:

Confidence(X -> Y) = Support_count(X ∪ Y) / Support_count(X)

Rule	Support	Confidence
(2^3)->5	2/4=50%	2/2=100%
(3^5)->2	2/4=50%	2/2=100%
(2^5)->3	2/4=50%	2/3=66%
2->(3^5)	2/4=50%	2/3=66%
3->(2^5)	2/4=50%	2/3=66%
5->(2^3)	2/4=50%	2/3=66%

Now remove the rules whose confidence is less than 70%(threshold confidence)

Final association rules generated are:

(2^3)->5
(3^5)->2

This will give us the relationship between the objects.

Advantages:

1. Simplicity & ease of implementation
2. The rules are easy to human-readable
3. Works well on unlabelled data
4. Flexibility & customisability
5. Extensions for multiple use cases can be created easily
6. The algorithm is widely used & studied

Disadvantages of Apriori algorithm:

1. Computational complexity: Requires many database scans.
2. Higher memory usage: Assumes transaction database is memory resident.
3. It needs to generate a huge no. of candidate sets.
4. Limited discovery of complex patterns

Improving the efficiency of Apriori Algorithm:

Here are some of the methods how to improve efficiency of apriori algorithm -

1. **Hash-Based Technique:** This method uses a hash-based structure called a hash table for generating the k-itemsets and their corresponding count. It uses a hash function for generating the table.
2. **Transaction Reduction:** This method reduces the number of transactions scanned in iterations. The transactions which do not contain frequent items are marked or removed.
3. **Partitioning:** This method requires only two database scans to mine the frequent itemsets. It says that for any itemset to be potentially frequent in the database, it should be frequent in at least one of the partitions of the database.
4. **Sampling:** This method picks a random sample S from Database D and then searches for frequent itemset in S. It may be possible to lose a global frequent itemset. This can be reduced by lowering the min_sup.
5. **Dynamic Itemset Counting:** This technique can add new candidate itemsets at any marked start point of the database during the scanning of the database.

Frequent Pattern-growth Algorithm

FP-growth is an algorithm for mining frequent patterns that uses a divide-and-conquer approach. FP Growth algorithm was developed by Han in 2000. It constructs a tree-like data structure called the frequent pattern (FP) tree, where each node represents an item in a frequent pattern, and its children represent its immediate sub-patterns. By scanning the dataset only twice, FP-growth can efficiently mine all frequent itemsets without generating candidate itemsets explicitly. It is particularly suitable for datasets with long patterns and relatively low support thresholds.

Working on FP Growth Algorithm

The working of the FP Growth algorithm in data mining can be summarized in the following steps:

Scan the database:

In this step, the algorithm scans the input dataset to determine the frequency of each item. This determines the order in which items are added to the FP tree, with the most frequent items added first.

Sort items:

In this step, the items in the dataset are sorted in descending order of frequency. The infrequent items that do not meet the minimum support threshold are removed from the dataset. This helps to reduce the dataset's size and improve the algorithm's efficiency.

Construct the FP-tree:

In this step, the FP-tree is constructed. The FP-tree is a compact data structure that stores the frequent itemsets and their support counts.

Generate frequent itemsets:

Once the FP-tree has been constructed, frequent itemsets can be generated by recursively mining the tree. Starting at the bottom of the tree, the algorithm finds all combinations of frequent item sets that satisfy the minimum support threshold.

Generate association rules:

Once all frequent item sets have been generated, the algorithm post-processes the generated frequent item sets to generate association rules, which can be used to identify interesting relationships between the items in the dataset.

FP Tree

The FP-tree (Frequent Pattern tree) is a data structure used in the FP Growth algorithm for frequent pattern mining. It represents the frequent itemsets in the input dataset compactly and efficiently. The FP tree consists of the following components:

Root Node:

The root node of the FP-tree represents an empty set. It has no associated item but a pointer to the first node of each item in the tree.

Item Node:

Each item node in the FP-tree represents a unique item in the dataset. It stores the item name and the frequency count of the item in the dataset.

Header Table:

The header table lists all the unique items in the dataset, along with their frequency count. It is used to track each item's location in the FP tree.

Child Node:

Each child node of an item node represents an item that co-occurs with the item the parent node represents in at least one transaction in the dataset.

Node Link:

The node-link is a pointer that connects each item in the header table to the first node of that item in the FP-tree. It is used to traverse the conditional pattern base of each item during the mining process.

The FP tree is constructed by scanning the input dataset and inserting each transaction into the tree one at a time. For each transaction, the items are sorted in descending order of frequency count and then added to the tree in that order. If an item exists in the tree, its frequency count is incremented, and a new path is created from the existing node. If an item does not exist in the tree, a new node is created for that item, and a new path is added to the tree. We will understand in detail how FP-tree is constructed in the next section.

Example:

Consider the following dataset with the transactions, assume min support count as 3:

Transaction ID	Items
T1	{M, N, O, E, K, Y}
T2	{D, O, E, N, Y, K}
T3	{K, A, M, E}
T4	{M, C, U, Y, K}
T5	{C, O, K, O, E, I}

Compute the frequency of each item:

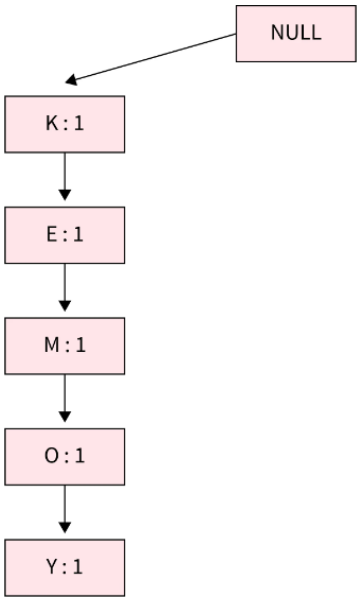
Item	Frequency
A	1
C	2
D	1
E	4
I	1
K	5
M	3
N	2
O	3
U	1
Y	3

Remove all the items below minimum support in the above table, we would remain with these items - {K: 5, E: 4, M : 3, O : 3, Y : 3}. Let's re-order the transaction database based on the items above minimum support. In this step, in each transaction, we will remove infrequent items and re-order them in the descending order of their frequency, as shown in the table below.

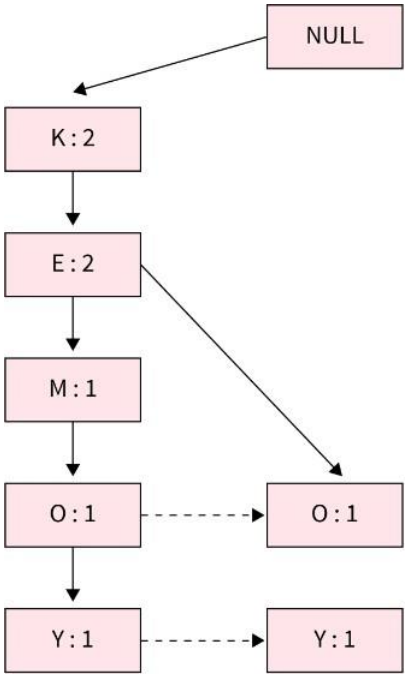
Transaction ID	Items	Ordered Itemset
T1	{M, N, O, E, K, Y}	{K, E, M, O, Y}
T2	{D, O, E, N, Y, K}	{K, E, O, Y}
T3	{K, A, M, E}	{K, E, M}
T4	{M, C, U, Y, K}	{K, M, Y}
T5	{C, O, K, O, E, I}	{K, E, O}

Now we will use the ordered itemset in each transaction to build the FP tree. Each transaction will be inserted individually to build the FP tree, as shown below -

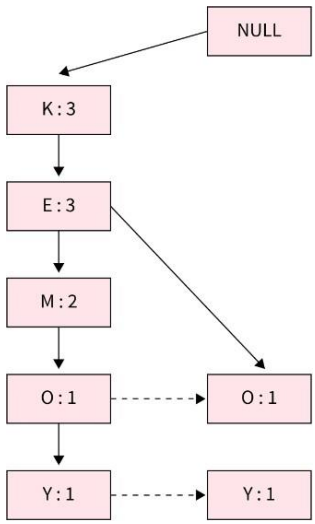
First Transaction {K, E, M, O, Y} : In this transaction, all items are simply linked, and their support count is initialized as 1.



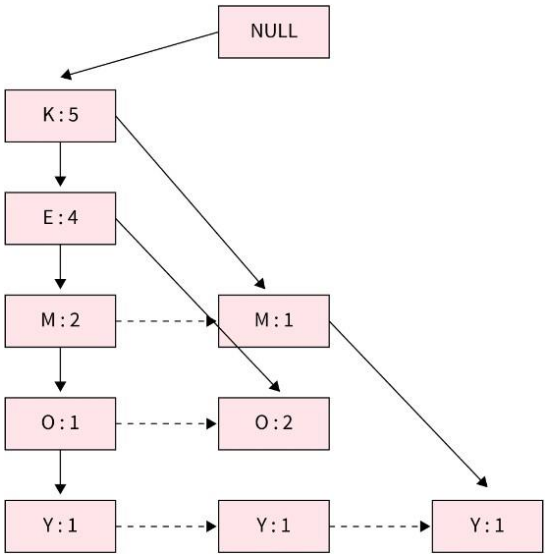
Second Transaction {K, E, O, Y} : In this transaction, we will increase the support count of K and E in the tree to 2. As no direct link is available from E to O, we will insert a new path for O and Y and initialize their support count as 1.



Third Transaction {K, E, M} : After inserting this transaction, the tree will look as shown below. We will increase the support count for K and E to 3 and for M to 2.



Fourth Transaction {K, M, Y} and Fifth Transaction {K, E, O} : After inserting the last two transactions, the FP-tree will look like as shown below:



Now we will create a Conditional Pattern Base for all the items. The conditional pattern base is the path in the tree ending at the given frequent item. For example, for item O, the paths {K, E, M} and {K, E} will result in item O. The conditional pattern base for all items will look like as shown below table:

Item	Conditional Pattern Base
Y	{K, E, M, O : 1}, {K, E, O : 1}, {K, M : 1}
O	{K, E, M : 1}, {K, E : 2}
M	{K, E : 2}, {K : 1}
E	{K : 4}
K	

Now for each item, we will build a conditional frequent pattern tree. It is computed by identifying the set of elements common in all the paths in the conditional pattern base of a given frequent item and computing its support count by summing the support counts of all the paths in the conditional pattern base. The conditional frequent pattern tree will look like this as shown below table:

Item	Conditional Pattern Base	Conditional FP Tree
Y	{K, E, M, O : 1}, {K, E, O : 1}, {K, M : 1}	{K : 3}
O	{K, E, M : 1}, {K, E : 2}	{K, E : 3}
M	{K, E : 2}, {K: 1}	{K : 3}
E	{K: 4}	{K: 4}
K		

From the above conditional FP tree, we will generate the frequent itemsets as shown in the below table:

Item	Frequent Patterns
Y	{K, Y - 3}
O	{K, O - 3}, {E, O - 3}, {K, E, O - 3}
M	{K, M - 3}
E	{K, E - 4}